



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Gerber Mariano Pérez Gómez **Carné:** 1165825

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable



Browser tabs: PORTAL ESTU... x MANEJO E IMPL... x My Information x Summary - Ove... x url-mia-2026/Se... x miastorageazu... x gmperezgom/s... x Pregunta a Gemini

github.com/gmperezgom/serviceconnector-webapp-storageblob-dotnet

gmperezgom / serviceconnector-webapp-storageblob-dotnet

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

serviceconnector-webapp-storageblob-dotnet Public

forked from Azure-Samples/serviceconnector-webapp-storageblob-dotnet

main 1 Branch 0 Tags

Go to file Add file Code

This branch is 1 commit ahead of Azure-Samples/serviceconnector-webapp-storageblob-dotnet:main

Contribute Sync fork

gmperezgom feat: agregar nombre al readme 51c51fd · 2 minutes ago 15 Commits

.github	.github/PULL_REQUEST_TEMPLATE.md committed	5 years ago
Pages	initial version	5 years ago
Properties	initial version	5 years ago
wwwroot	initial version	5 years ago
.gitignore	initial version	5 years ago
CHANGELOG.md	CHANGELOG.md committed	5 years ago
CONTRIBUTING.md	CONTRIBUTING.md committed	5 years ago

About

Sample of using Service Connector to connect Azure WebApp to Azure Storage Blob

Readme License Code of conduct Contributing Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

17°C Parc. soleado 7/10/2026

Browser tabs: PORTAL ESTU... x MANEJO E IMPL... x My Information x Summary - Ove... x url-mia-2026/Se... x miastorageazu... x Editing service... x Pregunta a Gemini

github.com/gmperezgom/serviceconnector-webapp-storageblob-dotnet/edit/main/README.md

gmperezgom / serviceconnector-webapp-storageblob-dotnet

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Files

main +

Go to file

.github Pages Properties wwwroot .gitignore CHANGELOG.md CONTRIBUTING.md Const.cs Program.cs README.md Startup.cs StorageHelper.cs WebStorageSample.csproj

serviceconnector-webapp-storageblob-dotnet

Edit Preview

```
1 Gerber
2 ---
3 page_type: sample
4 languages:
5   - csharp
6 products:
7   - azure
8   - service-connector
9 urlFragment: serviceconnector-webapp-storageblob-dotnet
10 ---
11
12 # Using Service Connector
13
14 The repository offers a sample application to create and verify a connection to Azure Storage using Service Connector.
15
16 ## Getting Started
17
18 ### 1. Prerequisites
```

Commit changes

Commit message

feat: agregar nombre al readme

Extended description

Add an optional extended description...

Message and description suggested by Copilot.

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Commit changes

Cancel changes Commit changes...

Spaces 2 Soft wrap

Use **Control + Shift + M** to toggle the **tab** key moving focus. Alternatively, use **esc** then **tab** to move to the next interactive element on the page.

Attach files by dragging & dropping, selecting or pasting them.

17°C Parc. soleado 7/10/2026



2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

3. Crear DotNet Env

a) En terminal, instalar: **dotnet add package DotNetEnv**

b) Crear .env file en raíz de proyecto, valores ->

AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/

AZURE_STORAGE_CONTAINER_NAME=default

c) Crear .env.production con los mismos valores en el mismo lugar

d) Anadir cambio brindado en program.cs

e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run

The screenshot shows the Visual Studio IDE with the following components:

- Explorer:** Displays the project structure for 'mia-semana14', including folders like 'serviceconnector-webapp-storageblob-dotnet' and files like '.env', '.env.production', and 'Program.cs'.
- Terminal:** Shows the command 'dotnet run' being executed. The output includes several warnings from CA2200 and CA2024, and information about the application starting and listening on ports 5001 and 5000.
- Output:** Displays the same terminal output as the terminal window.
- Chat:** A sidebar on the right with a 'Build with Agent' button and a description of the build process.

```
1 AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
2 AZURE_STORAGE_CONTAINER_NAME=default

PS C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet> dotnet run
package "Azure.Storage.Blobs" 12.10.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-64x4-9hc6-r2h6
C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet\StorageHelper.cs(34,17): warning CA2200: Si i
nicia de nuevo la excepción detectada, se cambia la información de la pila. (https://learn.microsoft.com/dot
net/fundamentals/code-analysis/quality-rules/ca2200)
C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet\StorageHelper.cs(66,17): warning CA2200: Si i
nicia de nuevo la excepción detectada, se cambia la información de la pila. (https://learn.microsoft.com/dot
net/fundamentals/code-analysis/quality-rules/ca2200)
C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet\StorageHelper.cs(54,33): warning CA2024: No u
sar 'StreamReader.EndOfStream' en un método asíncronico (https://learn.microsoft.com/dotnet/fundamentals/cod
e-analysis/quality-rules/ca2024)
warn: Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServer[8]
The ASP.NET Core developer certificate is not trusted. For information about trusting the ASP.NET Core
developer certificate, see https://aka.ms/aspnet/https-trust-dev-cert
info: Microsoft.Hosting.Lifetime[14]
Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet
info: Microsoft.Hosting.Lifetime[0]
Application is shutting down...
PS C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet> dotnet --version
10.0.401
PS C:\mia-semana14\serviceconnector-webapp-storageblob-dotnet>
```



4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → **tu Storage Account** → **Data storage** → **Containers**
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → **App Service** → **Settings** → **Identity** → **System assigned: on** → **Save (web app creada semana 13)**
- Configurar la conexión entre App Service y Blob Storage: **App Service** → **Settings** → **Service Connector** → **Create** → **Seleccionar "Storage Blob"** → **Seleccionar Storage Account [tu_blob_name]** → **Authentication "System-assigned managed identity"** → **Review + Create.**
- Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → **Settings** → **Endpoints** → **Blob service**

The screenshot displays the Azure portal interface. The top navigation bar shows the 'miastorageazure' web app selected under the 'Identity' section. The left-hand navigation pane lists various settings like 'Environment variables', 'Configuration', 'Instances', 'Authentication', 'Identity' (which is highlighted), 'Backups', 'Custom domains', 'Certificates', and 'Networking'. The main content area shows the 'System assigned' tab for the managed identity. It includes a 'Status' toggle set to 'On', the 'Object (principal) ID', and a 'Permissions' section with a link to 'Azure role assignments'. A blue information banner at the bottom of the permissions section provides a warning about Microsoft Entra ID registration.



Home > miastorageazure

miastorageazure | Service Connector

Web App

Using Service Connector will register resource provider 'Microsoft.ServiceLinker' to your Subscription.

+ Create Edit Refresh Validate Sample code Delete Feedback

What operations are done when creating a connection? [Learn more](#). Check code sample to use the connection in your application: [Sample code](#). Have feedback for Service Connector? [Fill out this quick survey](#)

Use Service Connector to connect your Azure Compute and target backing services. Swapping slots does not carry over service connectors or environment variables set by Service Connector. [Learn more](#)

Service type	Resource name	Connection name	Connection status
Storage - Blob	storagemialab72145545	storageblob_GP	Unvalidated

Add or remove favorites by pressing Ctrl+L+Shift+F

23°C Nublado

Buscar

ESP LAA

17:12 7/10/2026

6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`
- Verifique que pueda obtener un token para Azure Storage: `az account get-access-token --resource https://storage.azure.com/`
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> `dotnet run`
- Si hay error en acceso, ejecutar en terminal de vs code: `az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"`

Obtener info: `az account show --query "{subscriptionId:id, user:user.name}" -o table`
`az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table`
 g) Cerrar process `ctrl+c` y logear de nuevo: `az logout` y `az login`, Por ultimo `dotnet run`

Nota: no olvidar subir cambios a repo github fork



7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:

- main

pool:

vmImage: 'windows-latest'

variables:

buildConfiguration: 'Release'

dotnetVersion: '9.x'

Completar posteriormente

azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

1. Instalar .NET 9

- task: UseDotNet@2

displayName: 'Instalar .NET 9 SDK'

inputs:

packageType: 'sdk'

version: '\$(dotnetVersion)'

2. Restaurar dependencias

- task: DotNetCoreCLI@2

displayName: 'Restaurar dependencias'

inputs:

command: 'restore'

projects: '**/*.csproj'

3. Configurar ambiente de producción

- powershell: |

Copy-Item ".env.production" ".env" -Force

displayName: 'Configurar .env de producción'

4. Compilar

- task: DotNetCoreCLI@2

displayName: 'Compilar aplicación'

inputs:

command: 'build'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration) --no-restore'



```
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
    zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

The screenshot shows a web browser displaying a GitHub repository for 'gmperezgom/serviceconnector-webapp-storageblob-dotnet'. The file 'azure-pipelines.yml' is open, showing a YAML configuration for a CI/CD pipeline. The pipeline has a trigger for 'main' and a pool of 'windows-latest'. It includes variables for 'buildConfiguration' (Release) and 'dotnetVersion' (9.x). The steps section contains a task 'UseDotNet@2' with a display name 'Instalar .NET 9 SDK'. The task has inputs for 'packageType' (sdk) and 'package' (a path to a zip file). The interface includes a sidebar with file explorer, a top navigation bar with repository details, and a bottom taskbar with system icons and a search bar.

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

# Completar posteriormente
azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:
  # 1. Instalar .NET 9
  - task: UseDotNet@2
    displayName: 'Instalar .NET 9 SDK'
    inputs:
      packageType: 'sdk'
```



8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps son misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure Portal: Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save

The screenshot displays the Azure Portal interface for configuring a service connection. The browser address bar shows the URL: `dev.azure.com/url-2026-mia/MIA/_settings/adminservices?resourceId=18f530de-cfa4-4f31-97ad-76970fb4c3a5`. The left-hand navigation pane is open to 'Project Settings' for the 'MIA' project, with the 'Pipelines' section selected. The main content area shows the 'MIA_Connection' service connection details. The 'Overview' tab is active, displaying the service connection type as 'Azure Resource Manager using workload identity federation with openid connect'. The creator is identified as GERBER MARIANO PEREZ GOMEZ. Below this, the 'Workload Identity federation details' section shows the issuer URL and the subject identifier. The Windows taskbar at the bottom indicates a temperature of 22°C, a cloudy weather condition, and the time 17:55 on 7/10/2026.

9. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name

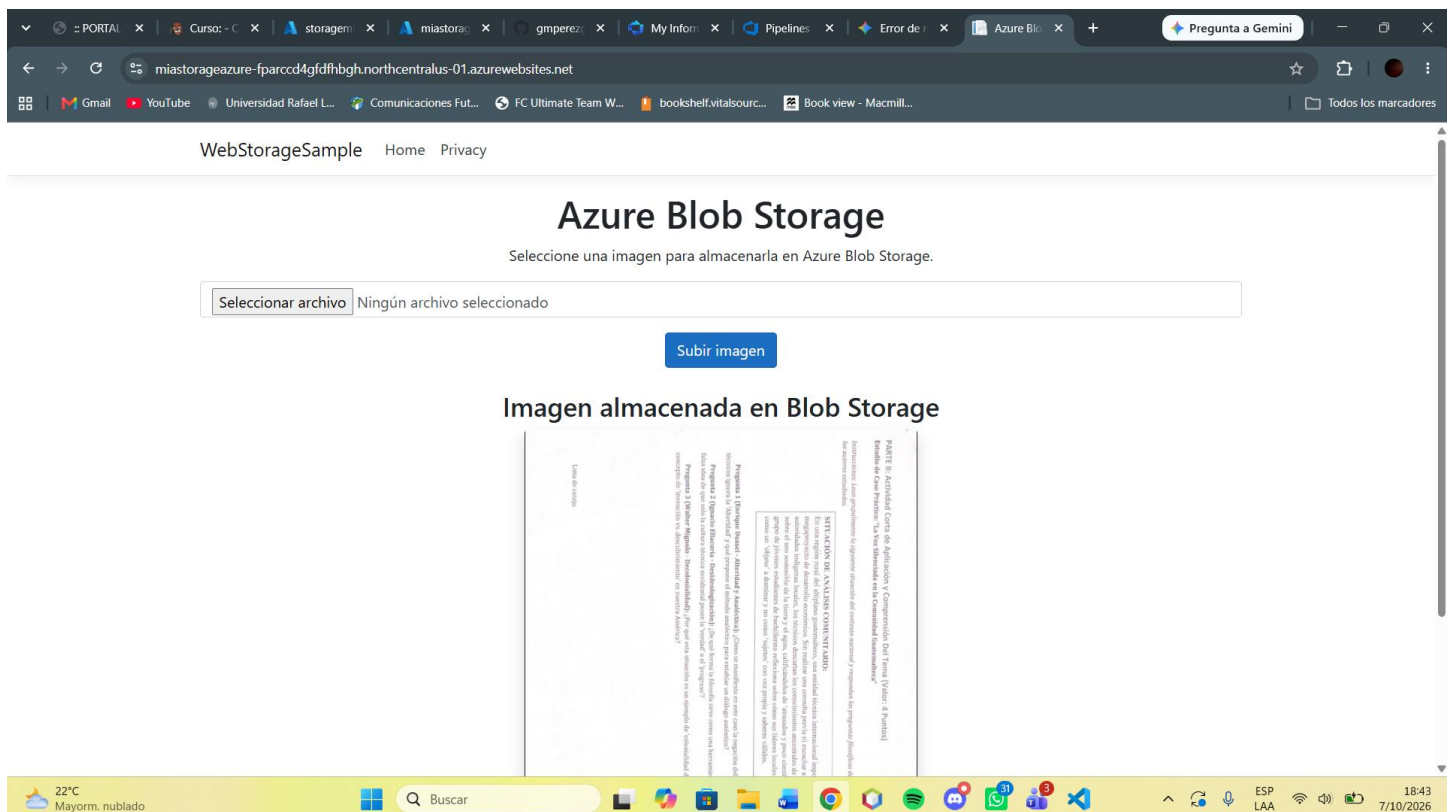
appName: Tu_app_name

- c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

10. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios





11. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?

El archivo .env guarda las variables de entorno que la aplicación necesita cuando corre en la máquina del desarrollador, por ejemplo el endpoint de Blob Storage que el código lee con `Environment.GetEnvironmentVariable(Const.ENDPOINT_ENV_KEY)`. Así la configuración no vive dentro del código y puede cambiar según el entorno. Casi siempre va en el .gitignore para que no llegue al repositorio. En Azure no se usa: allí la variable la inyecta el Service Connector o se define en los Application settings del App Service.

2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?

Guardar los archivos dentro del proyecto funciona hasta que la aplicación deja de depender de una sola máquina. El sistema de archivos del App Service puede perderse al reiniciar o redespargar, y si la app escala a varias instancias, cada una tiene sus propios archivos, de modo que lo que sube un usuario en una no lo ve la otra. Blob Storage está pensado para eso: los datos duran, están replicados y no cambian cuando se actualiza la aplicación. Además, separa los archivos del código, y eso permite redespargar sin riesgo de perder nada.

3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?

Son dos servicios distintos con direcciones distintas. El endpoint de Blob Storage (<https://<cuenta>.blob.core.windows.net/>) es la dirección del almacenamiento, y es a la que se conecta el código para subir y leer imágenes. La URL del App Service (<https://<app>.azurewebsites.net>) es la de la aplicación web, la que abre el usuario en el navegador. El usuario le habla al App Service y el App Service le habla a Blob Storage; el usuario nunca toca el endpoint directamente.

4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?

La Storage Account Key es una llave maestra: da acceso total a la cuenta, y si se filtra en un repositorio, un log o una captura de pantalla, la cuenta queda comprometida hasta que se rote. Con una identidad administrada no hay secreto que guardar, porque Azure le entrega a la app tokens temporales. También se le puede dar solo el permiso necesario, por ejemplo Storage Blob Data Contributor sobre esa cuenta, en lugar de acceso a todo. Por eso el código usa `DefaultAzureCredential` y no lleva ninguna clave.

5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?

GitHub guarda el código fuente y su historial. Azure DevOps Pipelines lo compila y lo despliega cada vez que hay cambios. La Service Connection es la autorización que le da a Azure DevOps permiso para actuar: le sirve para conectarse a la suscripción de Azure y desplegar en el App Service, y también para leer el repositorio de GitHub. Sin ella, el pipeline no tiene permisos para tocar nada.

6. ¿Qué ocurre después de hacer push a main?

Cuando se hace push a main, GitHub avisa a Azure DevOps y el pipeline arranca solo, como el Individual CI de la captura. Un agente descarga el código, restaura los paquetes, compila y publica el artefacto. Si el build pasa, usa la Service Connection para desplegarlo en el App Service, que se reinicia con la versión nueva. Si algo falla en la compilación, como el .cs sobrante que causó el error, el proceso se detiene ahí y no se despliega nada.



7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?

Un 403 con el pipeline en verde significa que el despliegue funcionó, pero la app no tiene permiso para escribir en Storage, así que el problema está en la autorización y no en el código. Lo primero es revisar que la identidad administrada del App Service tenga el rol Storage Blob Data Contributor, porque Owner o Contributor no dan acceso a los datos, y reiniciar la app por si el rol aún no se aplica. Después se revisan el firewall de la Storage Account y que la variable de endpoint apunte a la cuenta correcta. Si el contenedor ya existe, conviene crearlo a mano para no depender de `CreateIfNotExistsAsync()`. Para ubicar la causa, el Log stream lo dice: `AuthorizationPermissionMismatch` indica que falta el rol, y `AuthorizationFailure` apunta al firewall.

Entrega

Dentro del repositorio de GitHub, cree la carpeta `semana-14/lab1/` y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.